DOCUMENTAÇÃO TÉCNICA

surreal-namer

Como uma imagem se transforma em um nome: do pixel ao texto, com a matemática e cada decisão de projeto explicadas.



As 11 cores que o sistema sabe nomear (a classificação completa está na seção 3).

Revisão ampliada · novembro de 2026

Sumário

Como ler esta documentação 4

1. Visão geral 4

1.1 Duas interfaces, um núcleo 4

1.2 O princípio central: determinismo 5

1.3 O caminho de dados 5

1.4 O que influencia o nome? 5

2. Arquitetura do sistema 6

3. Análise de imagem (analyzer.py) 7

3.1 O contrato: analyze_image() 7

3.2 Cores dominantes 7

3.3 Do RGB ao nome da cor 8

3.4 Brilho 9

3.5 Proporção e orientação 9

3.6 EXIF 10

4. Vocabulário e gatilhos (namer.py) 11

4.1 Por que tuplas? 11

4.2 Substantivos 11

4.3 Adjetivos 12

4.4 Gatilhos contextuais 13

4.5 Quanto peso os gatilhos realmente têm? 14

5. Geração da seed 15

5.1 Passo 1: o texto de características 15

5.2 Passo 2: o hash MD5 16

5.3 Passos 3 e 4: oito dígitos viram um inteiro 16

5.4 O efeito avalanche, na prática 17

5.5 Colisões: o que significa “nome único”? 17

6. Seleção determinística de palavras 18

6.1 O resto da divisão: transformar um número gigante em uma posição válida 18

6.2 O deslocamento index × 1009 18

6.3 Notas de análise: o que o módulo não faz 19

6.4 Efeito colateral: qualquer mudança nas listas muda todos os nomes 19

7. Construção do nome final 19

7.1 Os passos, um a um, e o porquê . . . 20
7.2 Decidindo a estrutura com contas de resto . . . 20
7.3 O conector: só dois dos seis são alcançáveis . . . 21
7.4 As quatro estruturas possíveis . . . 21
7.5 Informações de depuração (verbose=True) . . . 22

8. Exemplos completos . . . 22
8.1 Exemplo A: foto horizontal (nome simples) . . . 22
8.2 Exemplo B: foto noturna vertical (nome com os dois extras) . . . 23
8.3 Gatilhos injetados em três cenários . . . 24

9. Interfaces . . . 24
9.1 CLI (main.py) . . . 24
9.2 GUI (gui.py) . . . 26

10. Build e distribuição . . . 27
10.1 Dependências . . . 27
10.2 Otimizações do build.spec . . . 27
10.3 Comandos . . . 27

11. Testes unitários . . . 28
11.1 test_analyzer.py (19 testes) . . . 28
11.2 test_namer.py (34 testes) . . . 28

12. Pontos de atenção e limitações . . . 28

13. Extensão e personalização . . . 29

Apêndice A. Correções em relação à documentação original . . . 30

Apêndice B. Glossário . . . 30

Apêndice C. Estrutura de arquivos . . . 31

Como ler esta documentação

Esta versão mantém todo o conteúdo técnico da documentação original e acrescenta, para cada etapa, **o que** ela faz, **como** a conta funciona e **por que** a decisão foi tomada assim. Quatro tipos de caixa aparecem ao longo do texto:

POR QUÊ
Explica a motivação de uma decisão de projeto: o problema que ela resolve e as alternativas descartadas.

MATEMÁTICA
Mostra uma conta passo a passo, com os números reais de um exemplo.

ATENÇÃO
Aponta limitações, armadilhas e inconsistências. Quando o ponto vem de uma análise desta revisão (e não da documentação original), isso é dito explicitamente.

EXEMPLO
Traz um exemplo executável com valores calculados de verdade.

ATENÇÃO · LEIA ANTES
Sobre a origem dos números. Os valores numéricos dos exemplos (strings, hashes MD5, seeds, restos de divisão, índices) foram calculados de verdade seguindo o algoritmo descrito na documentação original. As listas de palavras usadas nos exemplos são as listadas na seção 4, na ordem em que aparecem. Se a ordem ou o conteúdo das tuplas no código-fonte real for diferente, as *palavras* sorteadas serão outras; as contas e os índices, não.
Os pontos marcados como "análise desta revisão" decorrem da matemática do algoritmo descrito. Vale confirmá-los no código antes de agir sobre eles.

1. Visão geral

surreal-namer é um gerador de nomes surreais e desconexos. Ele analisa uma imagem (pixels e metadados EXIF) e produz um nome único e determinístico baseado nas características visuais dela. Uma foto horizontal em tons de azul e laranja, por exemplo, pode receber um nome como "Botao crepuscular": um substantivo do cotidiano combinado a um adjetivo sem relação aparente com ele. É essa desconexão que dá o tom surreal.

1.1 Duas interfaces, um núcleo

Interface	Arquivo	Uso típico	Saída
CLI	`main.py`	Terminal, scripts e automação, renomear lotes de imagens	Texto simples ou JSON
GUI	`gui.py`	Uso casual: arrastar uma imagem e ver o resultado	Janela PyQt6 com card de resultado

POR QUÊ
As duas interfaces são finas de propósito: só recebem a imagem, chamam o mesmo núcleo e mostram o resultado. Assim o nome é idêntico nas duas, e a lógica que importa (análise e geração) pode ser testada sem abrir janela nenhuma.

1.2 O princípio central: determinismo

Para a **mesma** imagem, o sistema **sempre** gera o **mesmo** nome. Não há aleatoriedade real, apenas pseudo-aleatoriedade derivada de um hash das características da imagem. Isso traz cinco ganhos concretos:

Ganho	Por que importa
Reprodutibilidade total	Rodar duas vezes a mesma imagem nunca produz dois nomes diferentes. Não existe estado escondido que mude entre execuções.
Consistência	Cópias idênticas da mesma foto, em pastas ou máquinas diferentes, recebem o mesmo nome.
Debug fácil	Dado o nome, é possível reconstruir o caminho inteiro (características → seed → palavras) e entender por que ele saiu.
Testabilidade	Testes unitários podem afirmar igualdade exata (“esta entrada deve gerar esta saída”), algo impossível com sorteio real.
Zero armazenamento	Não há banco de dados, contador ou arquivo de histórico. O nome é uma *função* da imagem: nome = f(características).

POR QUÊ · POR QUE NÃO USAR RANDOM?

Um sorteio verdadeiro (por exemplo, `random.choice`) devolveria um nome diferente a cada execução. Para repetir um resultado seria preciso guardá-lo em algum lugar. Ao derivar a “sorte” do próprio conteúdo da imagem, o sistema obtém variedade *entre* imagens e estabilidade *para cada* imagem.

ATENÇÃO

A contrapartida: não existe botão “gerar outro nome”. Para obter um nome diferente é preciso alterar a imagem, mesmo que minimamente (um pixel já basta, como a seção 5.4 demonstra).

1.3 O caminho de dados

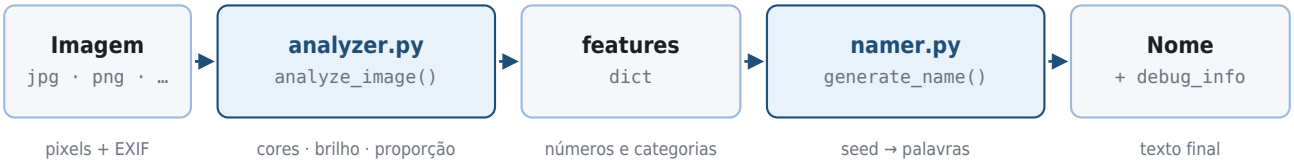


Figura 1. Fluxo de dados completo: a imagem vira um dicionário de características, e o dicionário vira um nome.

- **Imagem:** o arquivo fornecido pelo usuário (CLI ou GUI).
- `analyze_image()`**:** abre a imagem e mede cores, brilho, proporção e EXIF. Não sabe nada sobre nomes.
- **features:** um dicionário simples de números e categorias. É a *única* coisa que o próximo passo enxerga.
- `generate_name()`**:** transforma as características em uma seed e a seed em palavras. Não sabe nada sobre imagens.
- **Nome + debug_info:** o texto final e, no modo detalhado, todas as decisões intermediárias.

1.4 O que influencia o nome?

A imagem afeta o resultado de duas maneiras distintas: pela **seed** (que decide *qual* palavra é sorteada) e pelos **gatilhos** (que decidem *quais* palavras estão disponíveis para o sorteio). A tabela mostra o papel de cada característica:

Característica	Entra na seed?	Gera gatilhos?	Observação
Cores dominantes (nomes)	Sim, as 5 primeiras	Sim, as 3 primeiras → adjetivos	A ordem (por frequência) importa.
Brilho (valor 0–1)	Sim, com 4 casas decimais	Não (só a categoria gera)	

Característica	Entra na seed?	Gera gatilhos?	Observação
Categoria de brilho	Indiretamente (deriva do valor)	Sim → adjetivos	escura / média / clara
Proporção (largura/altura)	Sim, com 4 casas decimais	Sim → substantivos	Via categoria de proporção (4 faixas).
Orientação	Indiretamente (deriva da proporção)	Sim → substantivos	horizontal / vertical / quadrada
Largura × altura	Sim, valores exatos	Não	Separa imagens de mesma aparência e tamanhos diferentes.
EXIF	Não	Não	Apenas informativo.

2. Arquitetura do sistema

O projeto tem quatro módulos principais, com responsabilidades que não se sobrepõem:

Módulo	Responsabilidade	Depende de	Por que é separado
`analyzer.py`	Extração de características: cores, brilho, proporção, EXIF.	Pillow	Isola tudo que lê pixels. Pode ser trocado ou otimizado sem tocar na geração de nomes.
`namer.py`	Geração determinística de nomes a partir das características.	hashlib (biblioteca padrão)	Recebe só um dicionário: é uma função pura, testável com dados fabricados.
`main.py`	Ponto de entrada da CLI.	analyzer, namer	Interface fina para terminal e automação.
`gui.py`	Interface gráfica PyQt6 com drag-and-drop.	analyzer, namer, PyQt6	Interface fina para uso visual; o trabalho pesado roda em outra thread (seção 9.2).

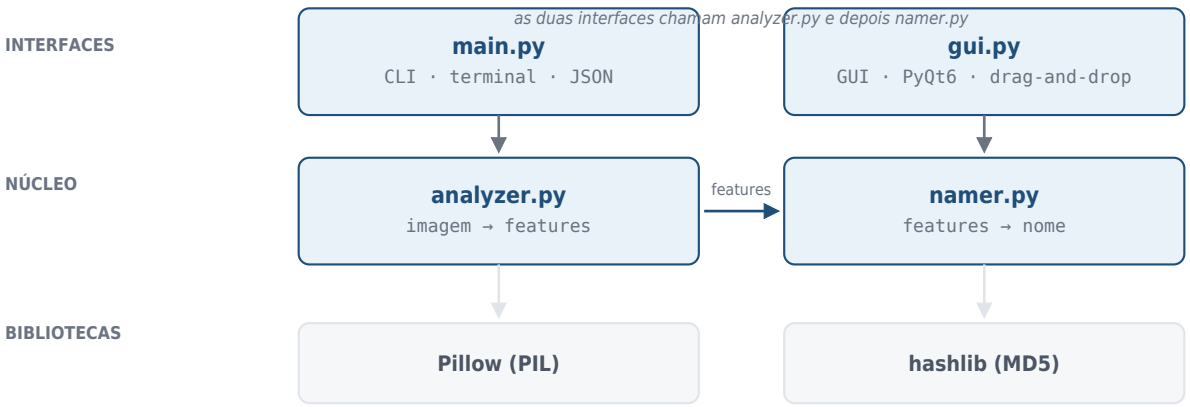


Figura 2. Camadas do projeto. As interfaces só orquestram; o núcleo concentra a lógica; as bibliotecas ficam nas pontas.

POR QUÊ

Fronteira estreita entre os módulos. O único dado que cruza de analyzer para namer é o dicionário de características. Por isso: (1) os testes do namer não precisam de imagens reais, bastam dicionários escritos à mão; (2) é possível mudar o jeito de medir uma imagem sem alterar a forma de nomear (cuidado: isso muda as seeds e, portanto, os nomes; ver seção 12); (3) as duas interfaces compartilham exatamente o mesmo comportamento.

3. Análise de imagem (analyzer.py)

3.1 O contrato: analyze_image()

`analyze_image(image_path) -> Dict[str, Any]` recebe o caminho de uma imagem e devolve um dicionário com os campos abaixo.

Campo	Tipo / faixa	Significado	Usado para
`width, height`	inteiros (pixels)	Dimensões da imagem.	Seed
`dominant_colors`	lista de tuplas RGB (máx. 5)	Cores dominantes, da mais para a menos frequente.	Só para gerar os nomes das cores
`color_names`	lista de textos	Nomes em português (“vermelho”, “azul”…).	Seed (5 primeiras) e gatilhos (3 primeiras)
`brightness`	0.0 a 1.0	Luminância média normalizada.	Seed
`brightness_category`	escura / media / clara	Faixa do brilho: < 0.3, 0.3–0.7, > 0.7.	Gatilhos
`aspect_ratio`	decimal	largura / altura.	Seed e gatilhos
`orientation`	horizontal / vertical / quadrada	> 1.3 horizontal, < 0.77 vertical, o resto quadrada.	Gatilhos
`exif`	dicionário	Metadados EXIF, se existirem.	Nada (informativo)

3.2 Cores dominantes

Descobrir “as cores de uma imagem” é um problema de redução: uma foto tem milhões de pixels e, na prática, dezenas de milhares de cores distintas. O algoritmo condensa isso em poucas cores representativas, em seis passos:

#	Passo	Por quê
1	Converte para RGB, se necessário.	Imagens podem chegar em modos como paleta, tons de cinza, RGBA ou CMYK. Padronizar em três canais evita casos especiais nos passos seguintes.
2	Reduz a imagem para um thumbnail de no máximo 150×150.	Desempenho. A paleta de uma foto não muda de forma relevante ao reduzi-la, mas o custo cai drasticamente (Figura 3).
3	Quantiza com MEDIANCUT para N cores (padrão 5).	Agrupa cores parecidas em N grupos. É o passo que transforma “milhares de cores” em “cinco cores”.
4	Extrai a paleta e a contagem de pixels de cada cor.	Saber *quanto* da imagem cada cor ocupa, e não só quais existem.
5	Ordena por frequência decrescente.	A ordem não é cosmética: as 5 primeiras cores entram na seed e as 3 primeiras geram gatilhos. A cor mais presente tem mais influência.
6	Retorna as N primeiras cores RGB.	Resultado final, já ordenado.

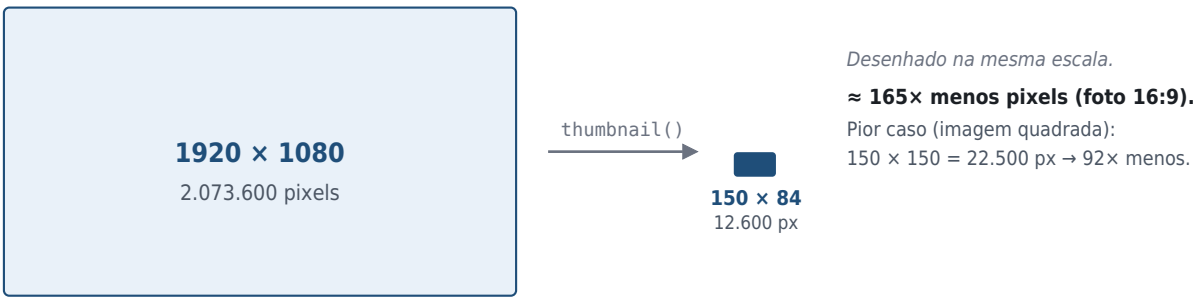


Figura 3. Por que reduzir a imagem antes de quantizar: o trabalho é proporcional ao número de pixels.

POR QUÊ

Por que MEDIANCUT? O método divide o espaço de cores em caixas: repetidamente corta, na mediana do canal (R, G ou B) de maior variação, a caixa que contém mais pixels, até sobrarem N caixas. A média de cada caixa vira uma cor da paleta. É rápido e, principalmente, **determinístico**: a mesma imagem sempre gera a mesma paleta. Alternativas populares como k-means dependem de um chute inicial aleatório, o que quebraria o princípio central do sistema.

3.3 Do RGB ao nome da cor

Cada cor RGB precisa virar uma palavra (“vermelho”, “azul”...). Para isso o código converte para **HSV** antes de decidir.

POR QUÊ

Por que HSV e não RGB? Em RGB, “azul claro” e “azul escuro” têm números bem diferentes nos três canais, e não há uma regra simples do tipo “é azul se...”. Em HSV a cor é separada em três grandezas independentes: **H (matiz, 0°–360°):** qual cor é, na roda de cores. **S (saturação, 0–1):** quão viva ela é (0 = cinza). **V (valor, 0–1):** quão clara ela é (0 = preto).
Com isso, “qual é a cor” vira uma pergunta só sobre H, e é possível tratar à parte os casos em que o matiz perde o sentido (cores muito escuras ou sem saturação).

As regras são avaliadas **na ordem abaixo**; a primeira que casar vence. Em código: `HSV = rgb_to_hsv(r, g, b)`, com `h_deg` sendo o matiz em graus.

Ordem	Condição	Resultado	Por que nesta posição
1	v < 0.2	preto	Em cores muito escuras o matiz é ruído: (10, 0, 12) e (0, 10, 2) parecem iguais ao olho, mas têm matizes muito diferentes.
2	v > 0.9 e s < 0.1	branco	Muito claro e quase sem saturação. Precisa vir antes do cinza, que capturaria o branco.
3	s < 0.15	cinza	Sem saturação, o matiz perde o sentido: todo cinza teria “H = 0” e viraria vermelho.
4	h_deg < 15 ou ≥ 345	vermelho	O vermelho cruza o 0°, por isso ocupa as duas pontas da roda.
5	h_deg < 45	laranja	
6	h_deg < 75	amarelo	
7	h_deg < 165	verde	Faixa larga: o olho distingue muitos tons de verde, que aqui são todos “verde”.
8	h_deg < 255	azul	Também larga: engloba ciano e azul.
9	h_deg < 285	roxo	
10	h_deg < 330	rosa	
11	caso contrário (330° a 345°)	vermelho	Regra de reserva: fecha a roda sem deixar buracos.

Matiz H (0°–360°): só é consultado quando a cor não é preto, branco nem cinza

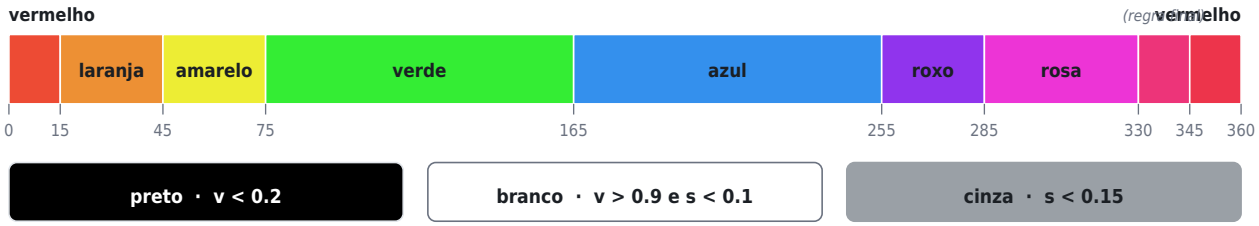


Figura 4. A roda de matizes cortada pelas regras 4 a 11, e as três regras de cor neutra que são checadas antes dela.

Exemplos calculados

RGB	H	S	V	Regra que casou	Nome
`(255, 0, 0)`	0°	1.00	1.00	H < 15 ou ≥ 345	**vermelho**
`(30, 144, 255)`	210°	0.88	1.00	H < 255	**azul**
`(255, 140, 0)`	33°	1.00	1.00	H < 45	**laranja**
`(139, 69, 19)`	25°	0.86	0.55	H < 45	**laranja**
`(128, 128, 128)`	0°	0.00	0.50	s < 0.15	**cinza**
`(245, 245, 245)`	0°	0.00	0.96	v > 0.9 e s < 0.1	**branco**
`(20, 20, 20)`	0°	0.00	0.08	v < 0.2	**preto**

ATENÇÃO

O “marrom” nunca é devolvido. Marrom é, fisicamente, um laranja (ou vermelho/amarelo) *escuro*. O marrom (139, 69, 19) da tabela acima tem H ≈ 25° e V ≈ 0,55: nenhuma regra o separa do laranja, então ele sai “laranja”. Os gatilhos de “marrom” (terra, café, chocolate...) existem no vocabulário, mas ficam inalcançáveis. Uma correção possível seria uma regra do tipo “H entre 15° e 45° e V < 0,6 → marrom”; ela mudaria as seeds de todas as imagens com tons terrosos.

3.4 Brilho

O brilho é um único número de 0.0 (preto absoluto) a 1.0 (branco absoluto): a **luminância média normalizada** de todos os pixels. A documentação original não especifica a fórmula de luminância; a mais usual pondera os canais pela sensibilidade do olho (verde pesa mais que azul): `L = 0.299·R + 0.587·G + 0.114·B`, dividida por 255. Confira a implementação em `get_brightness`.

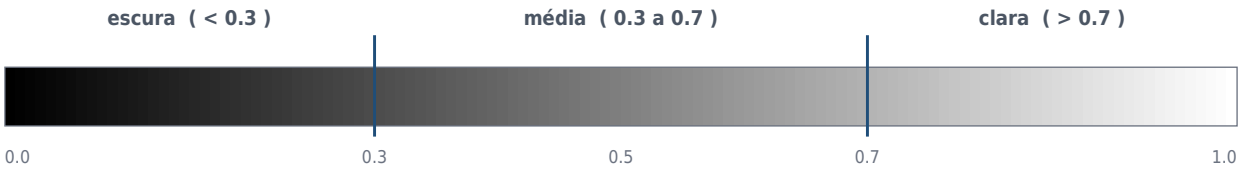


Figura 5. As três categorias de brilho e seus limiares.

POR QUÊ

Por que 0.3 e 0.7? Os limiares são simétricos em torno de 0.5 (0.5 ± 0.2): as faixas “escura” e “clara” têm a mesma largura, e a faixa “média” ocupa os 40% centrais. Três categorias bastam porque o brilho só serve aqui para escolher um pequeno conjunto de gatilhos (noturno, crepuscular, luminoso...).

ATENÇÃO

`get_brightness` percorre **todos** os pixels, ao contrário da análise de cores, que usa o thumbnail. Em imagens muito grandes isso pode ser lento. Usar o thumbnail também aqui seria mais rápido, mas mudaria levemente o valor do brilho e, com ele, a seed de toda imagem.

3.5 Proporção e orientação

A proporção é simplesmente **largura ÷ altura**. Dela saem duas classificações que usam limiares ligeiramente diferentes: a **orientação** (3 classes, devolvida por analyze_image) e a **categoria de proporção** (4 classes, usada só para escolher gatilhos).

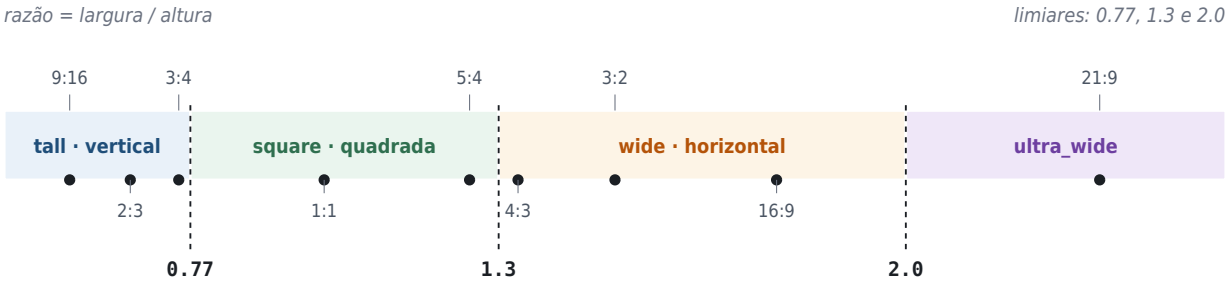


Figura 6. As quatro faixas de proporção e onde caem os formatos mais comuns (pontos em escala).

Formato	Razão	Orientação	Categoria (gatilhos)
1:1	`1.0000`	`quadrada`	`square`
5:4	`1.2500`	`quadrada`	`square`
4:5	`0.8000`	`quadrada`	`square`
4:3	`1.3333`	`horizontal`	`wide`
3:2	`1.5000`	`horizontal`	`wide`
16:9	`1.7778`	`horizontal`	`wide`
2:1	`2.0000`	`horizontal`	`wide`
21:9	`2.3333`	`horizontal`	`ultra_wide`
3:4	`0.7500`	`vertical`	`tall`
2:3	`0.6667`	`vertical`	`tall`
9:16	`0.5625`	`vertical`	`tall`

POR QUÊ

Por que 1.3 e 0.77? Note que 1 / 1.3 = 0.769…: os dois limiares são recíprocos, então uma foto deitada e a mesma foto em pé são tratadas de forma simétrica. A faixa central (0.77 a 1.3) é uma tolerância: formatos como 5:4 e 4:5 são “quase quadrados” a olho nu. Já 4:3 (1,333) cai logo acima de 1.3 e conta como horizontal.
E o “ultra_wide”? Panorâmicas e imagens de cinema (acima de 2.0) têm vocabulário próprio (panorâmico, cinemascope…), por isso a categoria de proporção tem uma faixa a mais que a orientação.

3.6 EXIF

O dicionário `exif` reúne os metadados da câmera (modelo, data, abertura…), quando existem. A leitura é básica e não cobre todos os formatos ou tags. **O EXIF não influencia a seed**: é apenas informativo.

POR QUÊ · RAZÕES PROVÁVEIS

A documentação original apenas afirma que o EXIF é informativo. As razões prováveis para isso estão no próprio desenho do sistema: (1) muitas imagens não têm EXIF, ou o perdem ao passar por mensageiros e redes sociais, então a seed mudaria de acordo com o caminho que o arquivo percorreu; (2) metadados podem ser editados sem alterar um único pixel, o que mudaria o nome sem que a imagem mudasse; (3) o nome pretende descrever o que se *vê*, e EXIF não é visual.

4. Vocabulário e gatilhos (namer.py)

4.1 Por que tuplas?

As listas de palavras são **tuplas** (sequências imutáveis e ordenadas) e os pools finais são concatenações de tuplas. Essa escolha é a base de todo o determinismo:

- **Ordem fixa.** A seleção é por posição (“a palavra de número 30”). Se a ordem pudesse variar, a mesma seed apontaria para palavras diferentes. Conjuntos (`set`) de textos, em Python, podem ser percorridos em ordens diferentes a cada execução; tuplas, não.
- **Imutabilidade.** Nenhuma parte do programa consegue alterar uma lista por acidente durante a execução.
- **Eficiência.** Concatenar tuplas (`A + B`) é simples e barato para pools desse tamanho.

> **ATENÇÃO**
> A ordem e o tamanho das tuplas fazem parte do “contrato” dos nomes: acrescentar, remover ou reordenar uma única palavra altera todos os nomes já gerados (seção 6.4).

4.2 Substantivos

A lista documentada tem **317 substantivos** em 48 categorias temáticas (o texto original citava “~200 itens”; a lista contabilizada aqui é maior). A mistura de universos (frutas, ferramentas, instrumentos musicais, fenômenos do céu, itens de casa) é o que gera combinações desconexas.

Categoria	Palavras
Frutas tropicais	melao, abacaxi, tangerina, jabuticaba, pitanga, caja, umbu
Plantas	cactus, samambaia, bromelia, orquidea, violetas, girassol
Construcao	tijolo, telha, cimento, areia, cal, argamassa, ladrilho
Cozinha	garfo, colher, faca, prato, xicara, pires, panela
Vestuario	meia, chinelo, cadarco, botao, ziper, bolso, gola
Ferramentas	prego, parafuso, porca, arruela, martelo, chave, serra
Natureza/ceu	nuvem, nevoa, orvalho, geada, granizo, chuvisco, arco-iris
Insetos	formiga, besouro, borboleta, libelula, grilo, cigarra, mosca
Moveis	cadeira, mesa, sofa, estante, tapete, cortina, almofada
Instrumentos	relogio, bussola, mapa, lente, espelho, oculos, binoculo
Musica	violino, flauta, tambor, pandeiro, triangulo, chocalho, berimbau
Escritorio	caderno, lapis, borracha, apontador, regua, esquadro, compassos
Calcados	botina, galocha, sapatilha, mocassim, tenis, sandalia, ralaste
Chuva	guarda-chuva, guarda-sol, capa, poncho, blusa, casaco, jaqueta
Cama	travesseiro, lencol, cobertor, edredom, fronha, colcha, manta
Acessorios	chaveiro, cordao, pulseira, anel, brinco, colar, broche
Limpeza	vassoura, rodo, balde, espanador, pa, enxada, ancinho
Arquitetura	janela, porta, fechadura, dobradica, batente, soleira, peitoril
Eletros	geladeira, fogao, forno, micro-ondas, liquidificador, batedeira, torradeira
Transporte	onibus, bonde, metro, trem, barca, balsa, gondola
Nautico	farol, boia, ancora, leme, vela, mastro, conves
Ingredientes	farinha, acucar, sal, fermento, ovos, leite, manteiga

Categoria	Palavras
Ferramentas metal	bigorna, tenaz, formao, goiva
Costura	agulha, linha, dedal, novelo, carretel, fita
Correspondencia	carimbo, pena, pergaminho, selo, envelope, carta
Luz	lanterna, fosforo, isqueiro, pavio, cera, goteira
Brinquedos	balao, pipa, catavento, bolha, piao, peteca, bexiga
Mesa	guardanapo, toalha, passadeira, capacho, esteira, tatame
Bebe	berco, carrinho, andador, cadeirinha, bebe-conforto, moisés, cercadinho
Bebidas	cafe, cha, suco, agua, refrigerante, cerveja
Padaria	pao, bolo, torta, biscoito, bolacha, rosca, sonho
Agua	praia, mar, rio, lago, cachoeira, fonte, nascentes
Relevo	montanha, morro, colina, vale, planicie, plato, vulcao
Espaco	estrela, planeta, lua, sol, cometa, asteroide, galaxia
Botanica	raiz, tronco, galho, folha, flor, fruto, semente
Anatomia	osso, dente, unha, cabelo, pele, sangue, musculo
Mental	pesadelo, devaneio, ideia, pensamento, memoria, esquecimento
Vento	vento, brisa, rajada, vendaval, furacao, tornado, ciclone
Geologia	pedra, rocha, seixo, cascalho, po, lama
Fogo	fogo, brasa, cinza, fumaca, faisca, chama, labareda
Metais	ouro, prata, bronze, cobre, ferro, aco, chumbo
Tecidos	seda, algodao, la, linho, couro, pelucia, veludo
Conservas	vinho, vinagre, azeite, mel, geleia, doce, compota
Leitura	livro, revista, jornal, gib quadrinho, manual, guia
Escrita	caneta, marcador, grafite, pincel, tela
Audio	fone, caixa, cabo, plug, tomada, interruptor, lampada
Computador	teclado, mouse, monitor, cpu, hd, placa-mae
Agua solida	gelo, neve, chuva

4.3 Adjetivos

São **251 adjetivos** em 38 categorias (o original citava “~140”). A lista mistura qualidades físicas (textura, temperatura), estados e emoções, e também termos de humor local, como os adjetivos de times de futebol brasileiros: um “tricolor” aplicado a um garfo é exatamente o tipo de desconexão buscada.

Categoria	Palavras
Times	flamenguista, vascaino, tricolor, alviverde, santista, corintiano, gremista
Estado	sonolento, acordado, desperto, adormecido, vigilante, distraido, atento
Textura	felpudo, liso, aspero, macio, duro, mole, escorregadio
Som	barulhento, silencioso, sussurrante, estridente, melodioso, dissonante, mudo
Temperatura	quente, frio, morna, gelada, escaldante, congelante, tepida
Velocidade	lento, rapido, devagar, apressado, parado, em movimento, estatico
Peso	leve, pesado, pena, chumbo, flutuante, afundando, pairando
Brilho	brilhante, opaco, fosco, lustroso, mate, cintilante, embacado

Categoria	Palavras
Sabor	doce, amargo, azedo, salgado, picante, insosso, temperado
Forma	redondo, quadrado, triangular, oval, retangular, hexagonal, informes
Idade	novo, velho, antigo, moderno, futurista, arcaico, atemporal
Tamanho	grande, pequeno, iminuto, gigante, microscopico, colossal, medio
Distancia	longe, perto, distante, proximo, adjacente, remoto, intimo
Altura	alto, baixo, rasteiro, elevado, suspenso, pendente
Umidade	seco, umido, molhado, encharcado, secando, evaporando, condensando
Limpeza	limpo, sujo, imaculado, manchado, puro, contaminado, higiênico
Cheiro	cheiroso, fedorento, perfumado, inodoro, aromatico, rançoso, fresco
Cor	colorido, monocromatico, vibrante, palido, intenso, desbotado, neon
Rigidez	rigido, flexivel, quebradiço, elastico, plastico, fragil
Vida	vivo, morto, inanimado, animado, respirando, pulsando
Liberdade	livre, preso, solto, amarrado, enjaulado, escravo, autonomo
Riqueza	rico, pobre, abastado, miseravel, opulento, escasso, farto
Emocao	triste, alegre, melancolico, eufórico, sereno, agitado, calmo
Forca	forte, fraco, poderoso, robusto, delicado, resistente
Inteligencia	sabio, tolo, inteligente, burro, genial, mediocre
Fase vida	jovem, idoso, anciao, crianca, adolescente, adulto, eterno
Utilidade	util, inutil, funcional, defeituoso, quebrado, inteiro, perfeito
Complexidade	simples, complexo, basico, avancado, primitivo, sofisticado, elementar
Raridade	comun, raro, unico, extraordinario, excepcional, singular, plural
Intensidade	suave, violento, brando, gentil
Luz	claro, escuro, luminoso, sombrio, radiantes, tenebroso, crepuscular
Bioma	arido, pantanoso, desertico, tropical
Ponta	pontiagudo, afiado, cego, curvo, reto, torto
Realidade	falso, verdadeiro, real, imaginario, ilusorio, concreto, abstrato
Movimento	movimento, dinamico, cinetico, potencial, inercial
Estado porta	aberto, fechado, entreaberto, trancado, destrancado, selado, lacrado
Ocupacao	cheio, vazio, lotado, deserto, povoado, habitado, abandonado
Uso	usado, gasto, novo em folha, segunda mao, vintage, retro

4.4 Gatilhos contextuais

Gatilhos são palavras **adicionais**, injetadas nos pools de acordo com as características da imagem. Eles expandem o vocabulário base com termos contextualmente relevantes: uma foto escura ganha “noturno” e “sombrio”; uma foto vertical ganha “torre” e “totem”.

> **POR QUÊ**
> **Por que cada gatilho vai para um pool específico?** Cores e brilho são *qualidades*, então viram adjetivos (“esmeralda”, “crepuscular”). Orientação e proporção sugerem *formas* e *objetos*, então viram substantivos (“torre”, “tapete”, “azulejo”).

Cor: COLOR_TRIGGERS (3 cores mais frequentes → pool de adjetivos)

Chave	Palavras injetadas
`vermelho`	flamenguista, corintiano, rubro-negro, sangrento, incandescente
`preto`	noturno, sombrio, obsidiano, ebano, carbonizado
`branco`	alvo, neve, leitoso, perolado, marfim
`azul`	tricolor, vascaino, oceanico, celeste, safira
`verde`	alviverde, gremista, esmeralda, musgoso, florestal
`amarelo`	canarinho, dourado, ambra, citrino, solar
`laranja`	tangerina, abobora, ferrugem, ambra, crepuscular
`roxo`	ametista, violeta, real, imperial, mistico
`rosa`	quartzo, rosado, corado, ruborizado, delicado
`cinza`	chumbo, prateado, cinzento, nevoento, metalico
`marrom`	terra, cafe, chocolate, caramelo, castanho

Orientação: ORIENTATION_TRIGGERS (→ pool de substantivos)

Chave	Palavras injetadas
`vertical`	tijolo, torre, coluna, obelisco, totem, monolito
`horizontal`	tapete, esteira, trilho, ponte, passarela, corredor
`quadrada`	quadro, azulejo, ladrilho, dado, cubo, bloco

Brilho: BRIGHTNESS_TRIGGERS (→ pool de adjetivos)

Chave	Palavras injetadas
`escura`	noturno, sombrio, obscuro, tenebroso, negro, eclipsado
`clara`	luminoso, radiante, ofuscante, esplendente, brilhante, solar
`media`	penumbra, crepuscular, meia-luz, suave, moderado, equilibrado

Proporção: ASPECT_TRIGGERS (→ pool de substantivos)

Chave	Condição	Palavras injetadas
`ultra_wide`	`>2.0`	panoramico, cinemascope, letterbox, widescreen, alongado
`wide`	`1.3-2.0`	retangular, alongado, estendido, comprido, horizontal
`tall`	`<0.77`	vertical, alto, estirado, alongado, torre
`square`	`0.77-1.3`	quadrado, perfeito, simetrico, equilibrado, geometrico

4.5 Quanto peso os gatilhos realmente têm?

Os gatilhos entram **no fim** das tuplas, e o sorteio é uniforme sobre o pool inteiro. Logo, a chance de uma palavra vir de um gatilho é simplesmente a fração do pool que eles ocupam. Com as listas documentadas, o pool de substantivos tem 317 + 6 + 5 = **328** palavras e o de adjetivos tem 251 + 15 + 6 = **272** (supondo três cores conhecidas).

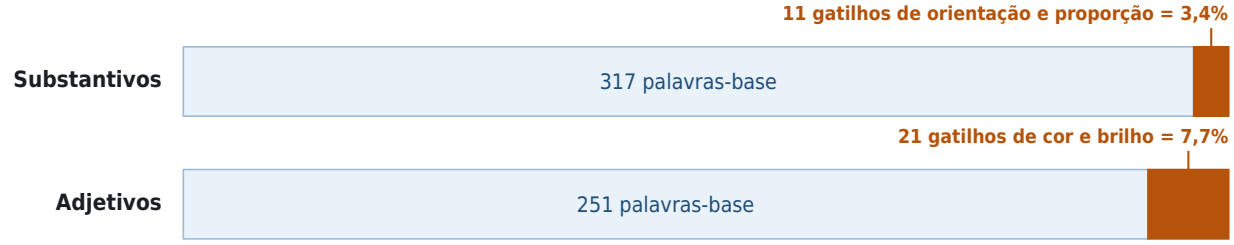


Figura 7. Composição dos pools. Os gatilhos (em laranja) ocupam uma faixa estreita.

MATEMÁTICA · ANÁLISE

Substantivos: 11 / 328 = 3,4%, Adjetivos: 21 / 272 = 7,7%.
Em média, só cerca de 1 substantivo em 30 e 1 adjetivo em 13 vem de um gatilho. **Análise desta revisão:** os gatilhos funcionam como tempero, não como prato principal; a influência dominante da imagem sobre o nome é a seed.

ATENÇÃO · PALAVRAS EM DOBRO

Algumas palavras existem **tanto no vocabulário base quanto nos gatilhos**. Como a concatenação de tuplas não remove repetidas, elas aparecem duas vezes no pool e têm o dobro de chance quando o gatilho está ativo. Substantivos repetidos: esteira, ladrilho, tapete, tijolo. Adjetivos repetidos: alviverde, brilhante, chumbo, corintiano, crepuscular, delicado, flamenguista, gremista, luminoso, real, sombrio, suave, tenebroso, tricolor, vascaino.

5. Geração da seed

A **seed** é o número que resume a imagem. Tudo o que vem depois (quais palavras, se há conector, se há segundo adjetivo) é calculado a partir dela. Ela é produzida por `get_seed_from_features(features) -> int` em quatro etapas: montar um texto, calcular o MD5, pegar 8 dígitos e converter para inteiro.

5.1 Passo 1: o texto de características

As características são concatenadas em um único texto no formato `cores|brilho|proporcao|dimensoes`. Com a imagem de exemplo A (foto horizontal 1920×1080):

```
azullaranjacinzapretobranco|0.5800|1.7778|1920x1080
```

Parte	Valor no exemplo	Formato	Por que está na seed
Cores	azullaranjacinzapretobranco	Nomes das 5 primeiras cores, colados, sem separador	Dá à seed a “identidade de cor” da imagem. A ordem importa: a cor mais frequente vem primeiro.
Brilho	0.5800	Decimal com 4 casas	Separa imagens de mesma paleta e luminosidade diferente.
Proporção	1.7778	Decimal com 4 casas	Distingue formatos (16:9, 4:3, 1:1…).
Dimensões	1920x1080	largura + “x” + altura	Separa imagens parecidas com resoluções diferentes, inclusive a mesma foto em tamanhos distintos.

POR QUÊ

Por que 4 casas decimais? Decimais "crus" são frágeis como texto: em ponto flutuante, 0.1 + 0.2 vale 0.30000000000000004, e uma pequena diferença de arredondamento entre duas máquinas geraria um texto (e uma seed) diferente. Fixar 4 casas faz com que valores numericamente iguais virem sempre o *mesmo* texto, ao custo de ignorar diferenças menores que 0,0001.

E os separadores "|"? Marcam a fronteira entre os campos, para que o fim das cores e o começo do brilho nunca se confundam.

5.2 Passo 2: o hash MD5

O texto passa por uma **função de hash**. O MD5 recebe qualquer texto e devolve 128 bits, normalmente escritos como 32 dígitos hexadecimais. Para o texto do exemplo:

```
md5 = 655b346ef97330b6da78c887d3018ef4
```

- **Determinismo:** o mesmo texto sempre produz o mesmo hash, em qualquer computador.
- **Embaralhamento (efeito avalanche):** qualquer mudança mínima no texto altera o hash de forma completa e imprevisível (seção 5.4). É isso que transforma características parecidas em seeds sem relação visível entre si.

POR QUÊ

Por que MD5 e não o `hash()` do Python? Por segurança de programação, o Python embaralha o `hash()` de textos de forma diferente a cada execução do programa. Usá-lo faria a mesma imagem receber nomes diferentes a cada vez que o programa abrisse. O MD5 da `hashlib` é definido por especificação (RFC 1321) e dá o mesmo resultado em qualquer máquina, sistema ou versão de Python.

E as falhas de segurança do MD5? São irrelevantes aqui. Ninguém está tentando forjar uma imagem para obter um nome específico; o hash serve apenas para embaralhar de forma estável e rápida.

5.3 Passos 3 e 4: oito dígitos viram um inteiro

Do hash de 32 dígitos, o código pega os **8 primeiros** e os lê como um número em base 16 (hexadecimal): `int(md5[:8], 16)`.

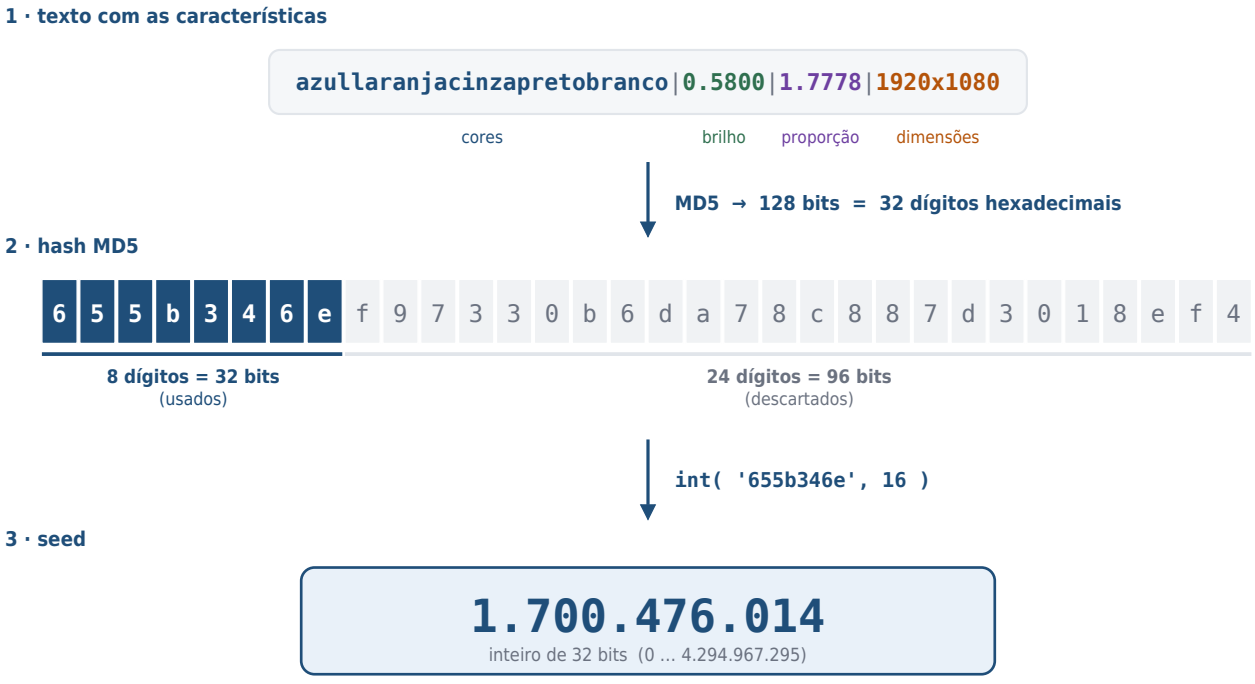


Figura 8. Da imagem à seed, com os valores reais do exemplo A.

A conversão, dígito por dígito

Posição	Dígito hex	Valor	Peso (16 elevado a k)	Contribuição
1	6	6	16^7 = 268.435.456	1.610.612.736
2	5	5	16^6 = 16.777.216	83.886.080
3	5	5	16^5 = 1.048.576	5.242.880
4	b	11	16^4 = 65.536	720.896
5	3	3	16^3 = 4.096	12.288
6	4	4	16^2 = 256	1.024
7	6	6	16^1 = 16	96
8	e	14	16^0 = 1	14
			Soma	**1.700.476.014**

MATEMÁTICA

Em base 16 cada posição vale 16 vezes a da direita (como em base 10 cada posição vale 10 vezes). Por isso o dígito mais à esquerda pesa 16^7 = 268.435.456, e a soma de todas as contribuições é a seed: **1.700.476.014**.
Oito dígitos hexadecimais ocupam exatamente 32 bits, então a seed está sempre entre 0 e $2^{32} - 1$ = 4.294.967.295.

POR QUÊ

Por que só 8 dígitos? (1) Basta: as listas têm algumas centenas de palavras, e 4,29 bilhões de valores possíveis sobram para sortear entre elas. (2) Um inteiro de 32 bits é compacto, fácil de exibir na GUI e de repetir em um relatório de debug. Os 24 dígitos descartados não fazem falta porque o MD5 espalha a informação de modo uniforme: qualquer pedaço do hash serve.

5.4 O efeito avalanche, na prática

A tabela parte do exemplo A e muda **uma única coisa** por vez. Repare que o nome muda por completo mesmo quando a mudança é minúscula:

Variação	Prefixo do MD5	Seed	Nome gerado
original	`655b346e`	1.700.476.014	**Botao crepuscular**
brilho 0.5800 → 0.5801	`0d1f54aa`	220.157.098	**Vale e verdadeiro**
largura 1920 → 1921	`3266c034`	845.594.676	**Peteca inteiro**
1ª e 2ª cor trocadas	`0c9906fd`	211.355.389	**Almofada de corintiano de fraco**

POR QUÊ

Esse comportamento é desejado: duas fotos “quase iguais” recebem nomes sem parentesco, o que reforça o caráter surreal e evita que o nome de uma imagem sirva de pista para o de outra. O custo é a ausência de proximidade: não existe “nome parecido para imagem parecida”.

5.5 Colisões: o que significa “nome único”?

A documentação original descreve os nomes como únicos. Em termos precisos, a **seed** tem 2^{32} valores possíveis, e o “paradoxo do aniversário” diz que, com valores uniformes, a chance de duas imagens distintas compartilharem a mesma seed chega a 50% perto de 77 mil imagens ($1{,}18 \times \sqrt{2^{32}} \approx 77.163$). Além disso, seeds diferentes podem gerar o mesmo nome, porque o espaço de nomes é bem menor que o espaço de seeds (seção 6.4). A unicidade é, portanto, **estatística**: muito provável em coleções pequenas, nunca garantida.

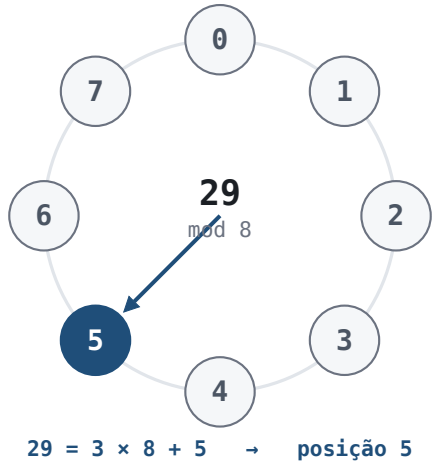
6. Seleção determinística de palavras

Com a seed pronta, é preciso escolher palavras de listas com centenas de itens. A função responsável é curta, mas concentra a matemática mais importante do projeto.

```
def select_from_list(lst, seed, index):
    combined_seed = (seed + index * 1009) % len(lst)
    return lst[combined_seed]
```

6.1 O resto da divisão: transformar um número gigante em uma posição válida

A seed é enorme (bilhões), mas a lista tem só algumas centenas de posições. O operador **%** (módulo) devolve o **resto da divisão** e resolve isso: dividir por N só pode deixar resto de 0 a N − 1, exatamente os índices válidos de uma lista com N itens.



Pense em um relógio de 8 posições. Partindo do 0 e andando 29 casas no sentido horário, você dá 3 voltas completas (3 × 8 = 24) e sobram 5 casas: o ponteiro para na posição 5.

É exatamente a conta **29 mod 8 = 5**. As voltas completas são descartadas; só o resto importa.

Com uma lista de N palavras, o relógio tem N posições: seja qual for o tamanho da seed, o ponteiro sempre cai em uma palavra que existe.

Figura 9. O resto da divisão como um ponteiro que dá voltas em uma lista circular.

> **MATEMÁTICA · DIVISÃO COM RESTO**
> Exemplo real: a seed do exemplo A é 1.700.476.014 e o pool de substantivos tem 328 palavras.
> 1.700.476.014 = 328 × 5.184.378 + **30** → posição **30** do pool → “**botao**”

6.2 O deslocamento index × 1009

Se o substantivo, o adjetivo e o conector usassem exatamente a mesma seed, as posições ficariam amarradas umas às outras de uma forma óbvia. O código afasta as escolhas somando a cada uma um deslocamento diferente: **index × 1009**. Além disso, a seed passada à função já recebe um pequeno acréscimo (+1, +4, +5). O deslocamento total de cada escolha é a soma dos dois:

Escolha	Chamada	Deslocamento total	Pool	Índice no exemplo A
Substantivo	`select(noun_pool, seed, 0)`	seed + 0	328	30
Adjetivo	`select(adj_pool, seed + 1, 1)`	seed + 1 + 1×1009 = seed + 1010	272	208
Conector	`select(connectors, seed + 4, 2)`	seed + 4 + 2×1009 = seed + 2022	6	0
2º adjetivo	`select(adj_pool, seed + 5, 3)`	seed + 5 + 3×1009 = seed + 3032	272	54

POR QUÊ

Por que 1009? A documentação original apresenta três razões: é um número primo próximo de 1000, distribui bem os índices 0 a 3 e evita colisões entre escolhas consecutivas. Do ponto de vista da matemática, a vantagem de um primo é que ele não tem divisores em comum com o tamanho de nenhuma lista (que é menor que 1009): assim, os múltiplos de 1009 nunca "travam" em poucas posições e percorrem todas elas, em vez de repetir um padrão curto.

6.3 Notas de análise: o que o módulo não faz

ATENÇÃO · PONTOS DE ATENÇÃO

★ **Análise desta revisão.** O deslocamento é uma constante: ele muda *onde* cada escolha cai, mas não elimina a relação matemática entre as escolhas. Com as listas documentadas, isso tem quatro consequências mensuráveis:

- **Substantivo e adjetivo não são independentes.** Os pools têm 328 e 272 palavras, e mdc(328, 272) = 8. Pelo teorema chinês do resto, só são alcançáveis 11.152 dos 89.216 pares possíveis (1 em 8).
- **O 2º adjetivo é sempre o adjetivo principal "empurrado" por uma distância fixa.** A diferença entre os dois deslocamentos é 3032 − 1010 = 2022, e 2022 mod 272 = 118. Logo, o 2º adjetivo é *sempre* a palavra 118 posições à frente do adjetivo principal na lista circular.
- **O espaço de nomes é limitado.** Todas as escolhas dependem apenas do resto da seed por mmc(328, 272, 12) = 33.456. Para esses tamanhos de pool existem no máximo 33.456 combinações, que resultam em 22.220 nomes distintos (verificado por enumeração).
- **Viés de uniformidade desprezível.** 2^{32} não é múltiplo de 328: 2^{32} mod 328 = 160, então as primeiras 160 posições aparecem uma vez a mais a cada ciclo. A diferença relativa é 1 / 13.094.412 ≈ $7{,}6 \times 10^{-8}$, irrelevante na prática.

6.4 Efeito colateral: qualquer mudança nas listas muda todos os nomes

Como a posição escolhida é *seed mod tamanho_do_pool*, alterar o **tamanho** de um pool muda o resto da divisão para todas as seeds. Acrescentar, remover ou até corrigir um erro de digitação que una ou separe duas palavras (como "gib quadrinho", na categoria Leitura) troca o nome de todas as imagens já nomeadas. Reordenar palavras sem mudar o tamanho produz o mesmo efeito, pois a mesma posição passa a apontar para outra palavra.

7. Construção do nome final

`generate_name(features, verbose=False) -> (nome, debug_info)` executa sete passos em sequência.



Figura 10. Os sete passos de generate_name(). As fórmulas são as da documentação original.

7.1 Os passos, um a um, e o porquê

Passo	O que faz	Por quê
1. Seed	Calcula a seed (seção 5).	Um único número que carrega a identidade da imagem para todo o restante.
2. Gatilhos	Consulta as quatro tabelas de gatilhos: cores (3 primeiras), orientação, brilho e proporção.	Aproxima o vocabulário disponível do que a imagem mostra.
3. Pools	noun_pool = SUBSTANTIVOS + gatilhos de orientação e proporção. adj_pool = ADJETIVOS + gatilhos de cor e brilho.	Concatenar (e não substituir) mantém o vocabulário base sempre disponível. Os gatilhos apenas ampliam.
4. Palavras-base	Sorteia o substantivo (índice 0) e o adjetivo (índice 1).	Todo nome tem, no mínimo, um substantivo e um adjetivo.
5. Estrutura	Decide, por aritmética sobre a seed, se haverá conector e se haverá 2º adjetivo.	Varia o formato dos nomes sem gastar aleatoriedade nova: tudo continua derivando da mesma seed.
6. Conector e 2º adjetivo	Se necessário, sorteia o conector (índice 2) e o 2º adjetivo (índice 3).	Só gasta o sorteio quando o elemento vai ser usado.
7. Montagem	Junta as partes, remove espaços das pontas e põe a 1ª letra em maiúscula. Se o resultado ficasse vazio, usaria “Sem Nome”.	Apresentação limpa. O “Sem Nome” é só uma proteção: como o substantivo nunca é vazio, na prática ele não é alcançado.

7.2 Decidindo a estrutura com contas de resto

```
use_connector  = (seed + 2) % 3 == 0
use_second_adj = (seed + 3) % 4 == 0
```

O programa não “sente” que um nome pede conector: ele apenas testa se um resto é zero. Simplificando as duas contas:

```
(seed + 2) % 3 == 0   ⇔   seed % 3 == 1
(seed + 3) % 4 == 0   ⇔   seed % 4 == 1
as duas ao mesmo tempo   ⇔   seed % 12 == 1
```

MATEMÁTICA

Probabilidades. Dividir por 3 deixa resto 0, 1 ou 2 com a mesma chance: o conector aparece em 1 de cada 3 seeds (33%). Dividir por 4 dá 4 restos possíveis: o 2º adjetivo aparece em 1 de cada 4 (25%).
Independência. 3 e 4 não têm divisor comum (são coprimos). Pelo teorema chinês do resto, o par (seed mod 3, seed mod 4) equivale a seed mod 12, e cada um dos 12 restos é igualmente provável. Logo as duas decisões são independentes: P(os dois) = 1/3 × 1/4 = 1/12.

resto de seed ÷ 12 (cada resto tem a mesma chance: 1 em 12)

0	1	2	3	4	5	6	7	8	9	10	11
—	C+A	—	—	C	A	—	C	—	A	C	—

— sem extras 6/12 = 50% | C só conector 3/12 = 25% | A só 2º adjetivo 2/12 ≈ 16,7% | C+A os dois 1/12 ≈ 8,3%

Figura 11. Os 12 restos possíveis de seed ÷ 12 e a estrutura de nome que cada um produz.

POR QUÊ

As taxas 1/3 e 1/4 são deliberadamente baixas: metade de todos os nomes (50%) fica em sua forma mais simples, “substantivo + adjetivo”, e as formas mais longas são a exceção. Isso mantém os nomes curtos e legíveis como nomes de arquivo.

7.3 O conector: só dois dos seis são alcançáveis

A tupla de conectores é `(' ', ' de ', ' do ', ' da ', ' e ', ' com ')`. O conector é escolhido pelo índice *(seed + 4 + 2 × 1009) mod 6 = (seed + 2022) mod 6*, mas apenas quando *use_connector* é verdadeiro.

ATENÇÃO · CONECTOR

★ **Análise desta revisão.** Quando *use_connector* é verdadeiro, vale seed ≡ 1 (mod 3). Como 2022 é divisível por 3 (2022 = 3 × 674), o índice do conector também satisfaz índice ≡ 1 (mod 3). Em uma lista de 6 posições, só os índices 1 e 4 cumprem isso.
Resultado: apenas **“ de ”** (índice 1) e **“ e ”** (índice 4) podem aparecer. **“ do ”**, **“ da ”**, **“ com ”** e o espaço simples da lista são inalcançáveis. Verificado por enumeração de seeds.
Correção possível: escolher o conector com outra fatia da seed, por exemplo `(seed // 3) % 6`, que não tem relação com o resto por 3. Isso mudaria os nomes de todas as imagens que usam conector.

7.4 As quatro estruturas possíveis

Como o mesmo conector é repetido antes de *cada* adjetivo, a estrutura completa é **substantivo + conector + 2º adjetivo + conector + adjetivo**. O texto original ilustrava isso com “Substantivo de Adjetivo1 e Adjetivo2”, mas o código nunca troca o conector entre os dois adjetivos.

substantivo · adjetivo	**50%**	`Sal prateado`
substantivo · de | e · adjetivo	**25%**	`Cercadinho e santista`
substantivo · 2º adjetivo · adjetivo	**16,7%**	`Fonte pontiagudo suspenso`
substantivo · de | e · 2º adjetivo · de | e · adjetivo	**8,3%**	`Algodao de contaminado de cinzento`

Figura 12. As quatro formas de nome, com a chance de cada uma e um exemplo real. Os quatro exemplos usam as mesmas cores e o mesmo brilho: só a resolução muda.

Resolução	Seed	seed mod 12	Conector?	2º adj.?	Nome
1000×2000	1.843.955.796	0	não	não	**Sal prateado**
1500×3000	2.249.926.690	10	sim	não	**Cercadinho e santista**
1080×2160	3.162.688.061	5	não	sim	**Fonte pontiagudo suspenso**
900×1800	3.200.670.853	1	sim	sim	**Algodao de contaminado de cinzento**

7.5 Informações de depuração (verbose=True)

No modo detalhado, a função devolve também um dicionário que permite auditar cada decisão:

Campo	Conteúdo
`seed`	Seed numérica calculada.
`color_triggers`	Gatilhos de cor aplicados.
`orientation_triggers`	Gatilhos de orientação.
`brightness_triggers`	Gatilhos de brilho.
`aspect_triggers`	Gatilhos de proporção.
`selected_noun`	Substantivo escolhido.
`selected_adj`	Adjetivo principal escolhido.
`use_connector`	Booleano: usou conector?
`use_second_adj`	Booleano: usou segundo adjetivo?
`connector`	Conector usado (ou None).

> **POR QUÊ**
> O debug existe por causa do determinismo: como todo resultado é reproduzível, todo resultado é também explicável. Qualquer nome pode ser auditado do começo ao fim, sem logs nem rastreamento adicional.

8. Exemplos completos

Dois exemplos percorrem o algoritmo inteiro, com todos os números reais. O exemplo A mostra um nome na forma mais simples (a mais comum). O exemplo B ativa os dois extras: conector e segundo adjetivo.

8.1 Exemplo A: foto horizontal (nome simples)

Uma foto 1920×1080 com cores dominantes azul, laranja, cinza, preto e branco, brilho 0,58 (categoria média), orientação horizontal e proporção “wide”.

Passo	Cálculo	Resultado
Características	cores azul, laranja, cinza, preto, branco · brilho 0.58 · 1920×1080	orientação horizontal · proporção wide · brilho media
1. Texto	cores + | + brilho + | + proporção + | + dimensões	`azullaranjacinzapretobranco|0.5800|1.7778|1920x1080`
2. MD5	md5(texto)	`655b346ef97330b6da78c887d3018ef4`
3. Seed	int('655b346e', 16)	**1.700.476.014**
4. Pools	substantivos: 317 base + 6 orientação + 5 proporção = 328 adjetivos: 251 base + 15 cor + 6 brilho = 272	
5. Substantivo	1.700.476.014 mod 328	índice 30 → **botao**
6. Adjetivo	(1.700.476.014 + 1010) mod 272	índice 208 → **crepuscular**
7. Conector?	(1.700.476.014 + 2) mod 3 = 2	não
8. 2º adjetivo?	(1.700.476.014 + 3) mod 4 = 1	não
Montagem	“botao” + “ crepuscular”	**Botao crepuscular**

EXEMPLO

Nome final: **Botao crepuscular**. Como (seed + 2) mod 3 não é zero e (seed + 3) mod 4 não é zero, o nome ficou na forma básica; o conector e o 2º adjetivo foram sorteados em potencial, mas não usados.

8.2 Exemplo B: foto noturna vertical (nome com os dois extras)

Corresponde ao “Exemplo 1” da documentação original: cores preto, azul e cinza, brilho 0,15 (escura), proporção 0,5 (vertical). A documentação não informa a resolução; foi adotada 900×1800 (razão exatamente 0,5), a primeira de uma pequena lista de resoluções comuns que ativa os dois extras.

Passo	Cálculo	Resultado
Características	cores preto, azul, cinza · brilho 0.15 · 900×1800	orientação vertical · proporção tall · brilho escura
1. Texto	cores + | + brilho + | + proporção + | + dimensões	`pretoazulcinza|0.1500|0.5000|900x1800`
2. MD5	md5(texto)	`bec65c85d21000d20599a0a66c2d5565`
3. Seed	int('bec65c85', 16)	**3.200.670.853**
4. Pools	substantivos: 317 base + 6 orientação + 5 proporção = 328 adjetivos: 251 base + 15 cor + 6 brilho = 272	
5. Substantivo	3.200.670.853 mod 328	índice 277 → **algodao**
6. Adjetivo	(3.200.670.853 + 1010) mod 272	índice 263 → **cinzento**
7. Conector?	(3.200.670.853 + 2) mod 3 = 0	sim
8. 2º adjetivo?	(3.200.670.853 + 3) mod 4 = 0	sim
9. Conector	(3.200.670.853 + 2022) mod 6 = 1	“ de ”
10. 2º adjetivo	(3.200.670.853 + 3032) mod 272	índice 109 → **contaminado**
Montagem	“algodao” + “ de contaminado” + “ de cinzento”	**Algodao de contaminado de cinzento**

EXEMPLO

Nome final: **Algodao de contaminado de cinzento**. Aqui seed mod 12 = 1, o único resto que ativa as duas decisões ao mesmo tempo (Figura 11).

8.3 Gatilhos injetados em três cenários

Os três cenários da documentação original, agora com a lista **completa** de gatilhos de cor (o original abreviava).

Foto noturna vertical (cores preto, azul, cinza; brilho · proporção = 0.15 · 0.5)

Gatilho	Palavras
Cor (adjetivos)	noturno, sombrio, obsidiano, ebano, carbonizado, tricolor, vascaino, oceanico, celeste, safira, chumbo, prateado, cinzento, nevoento, metalico
Orientação (substantivos)	tijolo, torre, coluna, obelisco, totem, monolito
Brilho (adjetivos)	noturno, sombrio, obscuro, tenebroso, negro, eclipsado
Proporção (substantivos)	vertical, alto, estirado, alongado, torre

Praia horizontal clara (cores amarelo, azul, branco; brilho · proporção = 0.85 · 1.8)

Gatilho	Palavras
Cor (adjetivos)	canarinho, dourado, ambra, citrino, solar, tricolor, vascaino, oceanico, celeste, safira, alvo, neve, leitoso, perolado, marfim
Orientação (substantivos)	tapete, esteira, trilho, ponte, passarela, corredor
Brilho (adjetivos)	luminoso, radiante, ofuscante, esplendente, brilhante, solar
Proporção (substantivos)	retangular, alongado, estendido, comprido, horizontal

Macro quadrada verde/terra (cores verde, marrom, cinza; brilho · proporção = 0.45 · 1.0)

Gatilho	Palavras
Cor (adjetivos)	alviverde, gremista, esmeralda, musgoso, florestal, terra, cafe, chocolate, caramelo, castanho, chumbo, prateado, cinzento, nevoento, metalico
Orientação (substantivos)	quadro, azulejo, ladrilho, dado, cubo, bloco
Brilho (adjetivos)	penumbra, crepuscular, meia-luz, suave, moderado, equilibrado
Proporção (substantivos)	quadrado, perfeito, simetrico, equilibrado, geometrico

> **ATENÇÃO**
> No cenário 3, o “marrom” aparece como cor de entrada, mas, como visto na seção 3.3, o classificador nunca devolve essa cor para imagens reais. O cenário só é alcançável fabricando as características à mão (como nos testes).

9. Interfaces

9.1 CLI (main.py)

Comando	Saída	Para quê
`python -m main imagem.jpg`	Apenas o nome gerado.	Uso em scripts: a saída limpa pode ser capturada direto para renomear arquivos.
`python -m main imagem.jpg --verbose`	Análise, gatilhos, seleção e resultado, em texto.	Entender por que um nome saiu.
`python -m main imagem.jpg --json`	Objeto JSON com nome, imagem e características.	Integração com outras ferramentas.
`python -m main imagem.jpg --verbose --json`	JSON com o campo extra “debug”.	Auditoria automatizada.

EXEMPLO · FORMATO DE SAÍDA

Os blocos a seguir mostram o **formato** das saídas descrito na documentação original, preenchido com os valores calculados do exemplo A. São ilustrativos: o texto exato do programa pode diferir em detalhes.

Saída padrão

```
Botao crepuscular
```

Saída --verbose

```
=== ANALISE DA IMAGEM ===
Dimensoes: 1920x1080
Proporcao: 1.78 (horizontal)
Brilho: 58% (media)
Cores dominantes: azul, laranja, cinza

=== GATILHOS IDENTIFICADOS ===
Cores: tricolor, vascaino, oceanico, celeste, safira, tangerina, abobora, ferrugem, ambra,
  crepuscular, chumbo, prateado, cinzento, nevoento, metalico
Orientacao: tapete, esteira, trilho, ponte, passarela, corredor
Brilho: penumbra, crepuscular, meia-luz, suave, moderado, equilibrado
Proporcao: retangular, alongado, estendido, comprido, horizontal

=== SELECAO DETERMINISTICA ===
Seed: 1700476014
Substantivo base: botao
Adjetivo base: crepuscular
Conector: ' ' (usado: False)
Segundo adjetivo: nao

=== RESULTADO ===
Nome: Botao crepuscular
```

Saída --json

```
{
  "name": "Botao crepuscular",
  "image": "imagem.jpg",
  "features": {
    "width": 1920,
    "height": 1080,
    "aspect_ratio": 1.7778,
    "orientation": "horizontal",
    "brightness": 0.58,
    "brightness_category": "media",
    "dominant_colors": [
      "azul",
      "laranja",
      "cinza"
    ]
  },
  "debug": "{ ... }  (apenas com --verbose)"
}
```

POR QUÊ

Por que três formatos? Cada um atende a um leitor diferente. O texto puro é para quem encadeia comandos; o JSON é para programas; o modo detalhado é para humanos investigando um resultado. Todos vêm da mesma chamada a generate_name(), então não existe divergência entre eles.

9.2 GUI (gui.py)

A interface gráfica usa PyQt6 e oferece:

- Drag-and-drop de arquivos de imagem e clique na área de soltura para abrir o seletor de arquivos.
- Pré-visualização da imagem carregada, redimensionada mantendo a proporção.
- Tema claro/escuro, com alternância por botão e persistência visual.
- Modo detalhado (caixa de seleção equivalente a `--verbose`).
- Barra de progresso animada durante o processamento.
- Card de resultado com o nome em destaque e a seed.
- Grade de características da imagem: dimensões, orientação, proporção, brilho e cores.
- Botão para copiar o nome para a área de transferência e botão para limpar e começar outra imagem.
- Barra de status com mensagens contextuais e animações sutis de pressionar/soltar nos botões.

Arquitetura thread-safe

O processamento roda em uma `WorkerThread` (subclasse de `QThread`) e se comunica com a interface por **sinais**: `finished(dict)`, `error(str)` e `progress(int, str)`.

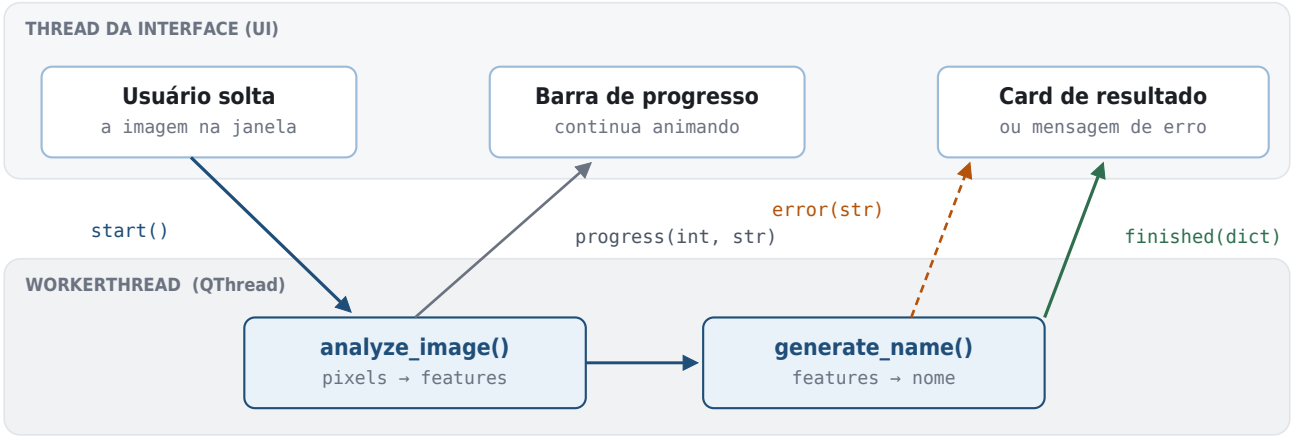


Figura 13. A janela nunca espera o cálculo: o trabalho acontece em uma thread separada e devolve resultados por sinais.

> **POR QUÊ**
> **Por que uma thread separada?** Interfaces gráficas têm uma única linha de execução responsável por desenhar a janela e reagir ao mouse. Se ela ficasse presa analisando uma imagem grande (lembre que o brilho percorre todos os pixels), a janela congelaria e o sistema operacional poderia marcá-la como "não respondendo". Com a análise em outra thread, a barra de progresso continua animada e a interface continua utilizável. Sinais são o mecanismo seguro do Qt para devolver o resultado: a thread de trabalho nunca mexe diretamente nos elementos da janela.

Temas

Claro: fundo azul-claro suave, texto escuro, acento azul-marinho. **Escuro:** fundo azul muito escuro, texto claro, acento azul-vivo. As cores ficam em dois dicionários (`LIGHT_COLORS` e `DARK_COLORS`) e o stylesheet QSS é gerado dinamicamente pelo `ThemeManager`.

> **POR QUÊ**
> Separar as cores em dicionários e gerar o QSS a partir deles evita manter duas folhas de estilo quase idênticas. Trocar de tema é apenas escolher outro dicionário e regenerar o texto do estilo; ajustar uma cor é mudar um valor em um só lugar.

10. Build e distribuição

10.1 Dependências

Tipo	Pacote	Função
Runtime	`Pillow>=10.0.0`	Processamento de imagem (abrir, reduzir, quantizar).
Runtime	`PyQt6>=6.0.0`	Interface gráfica.
Desenvolvimento	`pytest>=7.0.0`	Testes unitários.
Desenvolvimento	`PyInstaller>=6.0`	Empacotamento do executável.

As dependências de runtime ficam no `requirements.txt`; as de desenvolvimento são instaladas à parte, para que quem só usa o programa não precise delas.

10.2 Otimizações do build.spec

Opção	O que faz	Por quê
`strip=True`	Remove símbolos de debug dos binários.	Reduz o tamanho do executável; os símbolos só servem a depuradores.
`upx=True`	Comprime o executável com UPX.	Menos espaço em disco. Em contrapartida, a inicialização pode ficar um pouco mais lenta, e alguns antivírus desconfiam de binários comprimidos.
`console=False`	Sem janela de console (somente GUI).	Quem abre o programa com duplo clique não deve ver um terminal preto. Atenção: mensagens de erro impressas deixam de ser visíveis; para depurar, use console=True temporariamente.
excludes extensa	Remove módulos não usados (numpy, pandas etc.).	O PyInstaller empacota o que encontra no ambiente; excluir bibliotecas grandes que o projeto não usa evita inflar o executável.
hiddenimports	Garante os imports de PIL e PyQt6.	O PyInstaller descobre dependências lendo o código; imports feitos dinamicamente (como plugins de formatos de imagem) escapam dessa leitura e precisam ser declarados.

10.3 Comandos

```
python build_exe.py

# ou, manualmente:
pip install -r requirements.txt
pyinstaller build.spec --clean
```

A saída é `dist/surreal-namer.exe` (cerca de 42 MB). O tamanho se explica pelo fato de o executável embutir o interpretador Python e as bibliotecas Qt, que são grandes; por isso as otimizações acima valem a pena. A opção `--clean` descarta o cache de builds anteriores para evitar resíduos.

> **ATENÇÃO**
> O `build.spec` está configurado para Windows (`console=False`, saída .exe). Para Linux ou macOS é preciso ajustar o spec (por exemplo, `console=True` e remover o ícone .ico).

11. Testes unitários

O projeto tem **53 testes** (19 no analyzer e 34 no namer), executados com pytest.

```
python -m pytest test_analyzer.py test_namer.py -v
# 53 testes passam
```

11.1 test_analyzer.py (19 testes)

Função testada	O que se verifica
`rgb_to_hsv`	Conversão RGB → HSV para as cores primárias.
`get_color_name`	Classificação de cores: vermelho, verde, azul, preto, branco e cinza.
`get_aspect_ratio`	Casos quadrada, horizontal e vertical.
`get_brightness`	Preto = 0.0, branco = 1.0, cinza ≈ 0.5.
`get_dominant_colors`	Imagem de cor única.
`get_exif_data`	Imagem sem EXIF devolve dicionário vazio.
`analyze_image`	Integração completa: confere todos os campos do retorno.

11.2 test_namer.py (34 testes)

Função testada	O que se verifica
`get_seed_from_features`	Mesmas características → mesma seed; características diferentes → seed diferente.
`select_from_list`	Determinismo, índices diferentes e saída válida.
`get_color_triggers`	Gatilhos por cor e tratamento de cores desconhecidas.
`get_orientation_triggers`	Vertical, horizontal e quadrada.
`get_brightness_triggers`	Escura, clara e média.
`get_aspect_triggers`	ultra_wide, wide, tall e square.
`generate_name`	Devolve texto, é determinístico, começa com maiúscula e traz debug info.
Integridade dos dados	SUBSTANTIVOS e ADJETIVOS não vazios e sem repetição.

POR QUÊ
Os testes seguem o desenho do sistema: como o namer é uma função pura, basta fabricar dicionários de características e comparar com a saída esperada. O teste de “mesmas características → mesma seed” é o guardião do princípio central do projeto; se ele falhar, o determinismo foi quebrado.
Sugestão (análise desta revisão): acrescentar testes de regressão com “nomes dourados”: um conjunto fixo de características de referência e os nomes que elas devem gerar. Qualquer alteração de lista, de seed ou de índice faria esses testes falharem, avisando antes que todos os nomes mudem sem querer (ver seção 13).

12. Pontos de atenção e limitações

Os itens 1 a 6 vêm da documentação original, agora com a causa e as possíveis saídas. Os itens marcados com ★ são análises desta revisão, derivadas da matemática do algoritmo descrito (seções 4 a 7); confirme-os no código antes de agir.

#	Tema	O que acontece e por quê	Possível melhoria (e custo)
1	Determinismo absoluto	Mesma imagem = mesmo nome sempre. É o princípio de projeto (seção 1.2); a variação só vem de alterar a imagem.	Um parâmetro opcional de “sal” na string da seed permitiria variações. Muda nomes apenas quando usado.
2	Vocabulário fixo	As listas estão escritas no código (namer.py), assim como os gatilhos. Ampliar exige editar o código.	Mover listas para arquivo de dados. Atenção: mudar tamanho/ordem altera todos os nomes (seção 6.4).
3	Classificação de cores	Limiares fixos em HSV; cores de fronteira podem cair em vizinhas. “marrom” nunca é devolvido (é laranja escuro) e seus gatilhos ficam inalcançáveis.	Regra extra para marrom (H 15°–45° com V baixo). Muda seeds de imagens terrosas.
4	Desempenho	A análise de cores usa thumbnail; o brilho percorre todos os pixels. A GUI usa thread para não travar.	Calcular o brilho no thumbnail. Muda levemente o brilho e, portanto, as seeds.
5	EXIF	Leitura básica, não cobre todos os formatos. Não influencia a seed (seção 3.6).	Manter assim; EXIF não é visual.
6	Build só para Windows	O build.spec usa console=False e gera .exe.	Spec alternativo para Linux/macOS (console=True, sem .ico).
7 ★	Conector: só 2 de 6 alcançáveis	Por (seed + 2022) mod 6 e seed ≡ 1 (mod 3), só “ de ” e “ e ” aparecem (seção 7.3).	Escolher o conector com outra fatia da seed, ex.: (seed // 3) % 6. Muda nomes que usam conector.
8 ★	Escolhas correlacionadas e espaço de nomes limitado	Pelo teorema chinês do resto, substantivo e adjetivo são correlacionados (mdc = 8); o 2º adjetivo é o principal deslocado de 118 posições; há só 22.220 nomes distintos para pools de 328 e 272 palavras (seção 6.3).	Derivar cada escolha de uma fatia independente do hash (por exemplo, de bytes diferentes do MD5). Muda todos os nomes.
9 ★	Gatilhos pesam pouco	Só ≈ 3,4% dos substantivos e ≈ 7,7% dos adjetivos vêm de gatilhos (seção 4.5).	Aumentar a lista de gatilhos ou dar a eles mais peso. Muda nomes das imagens afetadas.
10 ★	Unicidade é estatística	Seeds de 32 bits colidem por volta de 77 mil imagens; nomes colidem antes, pelo espaço menor.	Usar mais dígitos do MD5 e mais palavras. Muda todos os nomes.

13. Extensão e personalização

Antes de qualquer alteração, uma regra geral: **o nome de uma imagem é função de três coisas — a string da seed, as contas de índice e o tamanho/ordem das listas.** Mexer em qualquer uma delas muda os nomes já gerados.

Ação	Como fazer	Efeito nos nomes existentes
Adicionar substantivos	Editar a tupla SUBSTANTIVOS em namer.py, mantendo ordem alfabética ou temática. Garantir que não haja repetidas (o teste de integridade confere).	Altera o tamanho do pool: **todos** os nomes mudam.
Adicionar adjetivos	Editar a tupla ADJETIVOS em namer.py, com a mesma regra.	Altera o tamanho do pool: **todos** os nomes mudam.
Novos gatilhos de cor	Adicionar uma entrada em COLOR_TRIGGERS. A chave é o nome devolvido por get_color_name(); o valor é uma tupla de textos.	Muda apenas imagens que tenham essa cor entre as 3 principais.
Nova categoria de gatilho	1) Criar o dicionário NOVA_CATEGORIA_TRIGGERS. 2) Criar get_nova_categoria_triggers(). 3) Injetar no pool correspondente em generate_name(). 4) Incluir no debug_info quando verbose.	Muda o tamanho de um pool: **todos** os nomes mudam.
Mudar o algoritmo da seed	Modificar get_seed_from_features().	**Todos** os nomes mudam. A documentação original já alerta para essa quebra de compatibilidade.

Ação	Como fazer	Efeito nos nomes existentes
Mudar a estrutura do nome	Modificar a lógica de generate_name() após a seleção (prefixos, sufixos, ordem...).	Muda o formato dos nomes; as palavras sorteadas continuam as mesmas.

POR QUÊ

A documentação original afirma que "a tupla já previne duplicatas automaticamente". Isso não é verdade: tuplas aceitam elementos repetidos. É o teste de integridade ("sem repetição") que protege as listas, então ele deve ser rodado após qualquer edição.

Boa prática: antes de editar, gere os nomes de um conjunto fixo de imagens e guarde o resultado. Depois da edição, compare. Se a mudança é intencional (e vai alterar nomes), isso fica registrado; se não era, os testes de regressão avisam.

Apêndice A. Correções em relação à documentação original

Ao recalcular os exemplos, algumas passagens da documentação original se mostraram inconsistentes com o algoritmo descrito nela mesma. Elas foram corrigidas nesta revisão:

#	No original	Correção / observação
1	Exemplo da seed: texto "vermelhopretbranco|0.4200|1.5000|1920x1080".	Faltava o "o" de "preto". O texto correto é vermelhopretobranco|0.4200|1.5000|1920x1080.
2	Mesmo exemplo: "md5 = a1b2c3d4e5f67890..." e seed = 2712848340.	Valores ilustrativos que não correspondem ao texto. O MD5 real é b7f5e3cffa636267... e a seed real é **3.086.345.167**.
3	Exemplo do modo --verbose: seed 2712848340, "Conector: ' de ' (usado: True)" e "Segundo adjetivo: sim".	Pela fórmula, (2712848340 + 2) mod 3 = 2 e (2712848340 + 3) mod 4 = 3, ambos diferentes de zero: essa seed não usa conector nem 2º adjetivo. O exemplo era ilustrativo.
4	Estruturas: "Substantivo de Adjetivo1 e Adjetivo2".	O código repete o *mesmo* conector antes de cada adjetivo ("Substantivo de Adj2 de Adj1") e só " de " e " e " são alcançáveis (seção 7.3).
5	Índices 4 e 5 "(reservado)".	Os índices passados à função são 0, 1, 2 e 3. Os números 4 e 5 aparecem como acréscimos à seed (seed + 4, seed + 5), não como índices.
6	"SUBSTANTIVOS (~200 itens)" e "ADJETIVOS (~140 itens)".	As listas documentadas têm 317 e 251 itens.
7	"A tupla já previne duplicatas automaticamente."	Tuplas aceitam repetições; a unicidade é garantida apenas pelo teste de integridade.
8	Possíveis erros de digitação no vocabulário.	"gib quadrinho" (provavelmente "gibi" e "quadrinho"), "ralaste", "compassos", "violetas"; e acentos pontuais (moisés, eufórico, higiênico, rançoso, quebradiço) em listas sem acentuação. Corrigi-los altera tamanho/ordem das listas e, portanto, os nomes.

Apêndice B. Glossário

Termo	Significado
Determinismo	Mesma entrada, sempre a mesma saída.
Seed	Número inteiro de 32 bits derivado das características da imagem; origem de todas as escolhas.
Hash (MD5)	Função que transforma qualquer texto em 128 bits de aparência aleatória, mas sempre iguais para o mesmo texto.
Efeito avalanche	Mudança mínima na entrada de um hash causa mudança total na saída.

Termo	Significado
Módulo (%)	Resto da divisão. N % K devolve um valor de 0 a K − 1.
Pool	Lista final de palavras disponíveis para sorteio: vocabulário base + gatilhos.
Gatilho (trigger)	Palavra extra injetada no pool conforme uma característica da imagem.
HSV	Modelo de cor com matiz (H), saturação (S) e valor (V).
Quantização	Redução de milhões de cores a poucas cores representativas.
MEDIANCUT	Método de quantização que divide o espaço de cores recursivamente pela mediana.
EXIF	Metadados gravados pela câmera dentro do arquivo de imagem.
Teorema chinês do resto	Resultado que relaciona restos por números coprimos: (seed mod 3, seed mod 4) equivale a seed mod 12.
mdc / mmc	Máximo divisor comum / mínimo múltiplo comum.

Apêndice C. Estrutura de arquivos

```
surreal-namer/
├── analyzer.py        # Análise de imagem (cores, brilho, proporção, EXIF)
├── namer.py           # Geração determinística de nomes (núcleo)
├── main.py            # CLI entry point
├── gui.py             # Interface gráfica PyQt6
├── build_exe.py       # Script de build automatizado
├── build.spec         # Config PyInstaller
├── requirements.txt   # Dependências runtime
├── test_analyzer.py   # Testes unitários analyzer
├── test_namer.py      # Testes unitários namer
├── dist/
│   └── surreal-namer.exe   # Executável final (após build)
├── build/             # Artefatos temporários PyInstaller (ignorar)
├── __pycache__/       # Bytecode Python (ignorar)
├── .pytest_cache/     # Cache pytest (ignorar)
└── DOCUMENTACAO.txt   # Documentação original
```

— fim da documentação —